

Informe Explicativo

API REST "Planetas" (BackEnd)

1. Descripción general

Este proyecto es una API REST construida con Node.js y Express, que permite Registrar, Ver, Modificar y Eliminar datos de una tabla llamada "Planetas", almacenada en una base de datos SQLite.

La arquitectura sigue un patrón simple de Router → Controller → Base de Datos:

```
index.js → Rutas.routes.js → Planetas.Controller.js → database.js (SQLite)
```

2. Estructura del proyecto

```

BackEnd/
├── src/
│   ├── Controller/
│   │   └── Planetas.Controller.js → Lógica de negocio (CRUD)
│   ├── db/
│   │   ├── data.db → Archivo físico de la base de datos SQLite
│   │   └── database.js → Conexión y creación de la tabla
│   └── Router/
│       └── Rutas.routes.js → Definición de endpoints
├── .env → Variables de entorno (ej. PORT)
├── index.js → Punto de entrada del servidor
├── package.json
└── package-lock.json
  
```

3. Dependencias necesarias e instalación

Antes de instalar nada, se necesita tener Node.js (incluye npm) instalado en el sistema. Se puede verificar con:

```
node -v
npm -v
```

3.1 Iniciar el proyecto (si aún no existe package.json)

```
npm init -y
```

3.2 Dependencias utilizadas en el código

Dependencia	Uso en el proyecto	Comando de instalación
express	Framework para crear el servidor y las rutas HTTP	npm install express
sqlite3	Motor de base de datos embebido (crea/gestiona data.db)	npm install sqlite3
cors	Habilita solicitudes desde otros dominios (ej. el FrontEnd)	npm install cors
dotenv	Permite leer variables de entorno desde un archivo .env	npm install dotenv

3.3 Instalación en un solo paso

```
npm install express sqlite3 cors dotenv
```

3.4 Opcional (recomendado para desarrollo): nodemon

Permite que el servidor se reinicie automáticamente al guardar cambios:

```
npm install --save-dev nodemon
```

Y agregar en package.json:

```
"scripts": {  
  "dev": "nodemon index.js",  
  "start": "node index.js"  
}
```

3.5 Archivo .env

Crear un archivo .env en la raíz del BackEnd con al menos:

```
PORT=3001
```

4. Cómo levantar el servidor

```
cd BackEnd  
npm install  
node index.js
```

Si todo está correcto, en la consola aparecerá:

```
 http://localhost:3001
```

5. Explicación del código, archivo por archivo

5.1 index.js — Punto de entrada

- Importa express, cors y dotenv.
- dotenv.config() carga las variables del archivo .env.
- Define el puerto: usa process.env.PORT o, si no existe, el 3001 por defecto.
- App.use(cors()) permite peticiones desde otros orígenes (por ejemplo, el FrontEnd en otro puerto).
- App.use(express.json()) permite leer el body de las peticiones en formato JSON.
- Importa el archivo de rutas (Rutas.routes.js) y lo monta con App.use('/', Rutas).
- App.listen(PORT, ...) inicia el servidor y muestra la URL en consola.

5.2 database.js — Conexión y creación de la base de datos

- Importa sqlite3 en modo verbose (mensajes de error más detallados) y el módulo path.
- Crea (o abre, si ya existe) el archivo físico ./src/db/data.db mediante new sql.Database(...).
- Si la conexión falla, se muestra un error por consola; si es exitosa, se ejecuta CREATE TABLE IF NOT EXISTS Planetas (...) que crea la tabla solo si no existe.
- Columnas: ID_Planeta (entero, clave primaria autoincremental), Nombre (texto, obligatorio), Descripcion (texto, obligatorio), Dis (texto, obligatorio — probablemente 'Distancia').
- module.exports = ConexionBD expone la conexión para que otros archivos (como el Controller) puedan usarla.

5.3 Planetas.Controller.js — Lógica de negocio (CRUD)

Cada función maneja una operación distinta sobre la tabla Planetas:

• RegistroDatos(req, res) — Crear (Create)

- Extrae Nombre, Descripcion y Dis del cuerpo (req.body) de la petición.
- Ejecuta INSERT INTO Planetas (...) VALUES (?, ?, ?) usando parámetros preparados (evita inyección SQL).

- Si falla → responde 500 con un mensaje de error.
- Si tiene éxito → responde 201 con un mensaje de confirmación.

• VerDatos(req, res) — Leer (Read)

- Ejecuta `SELECT * FROM Planetas` mediante `ConexionBD.all(...)`, que devuelve todas las filas.
- Si falla → responde 500.
- Si tiene éxito → responde 201 con la tabla completa de resultados.

Nota: para una lectura exitosa lo habitual sería devolver el código 200 OK en vez de 201 (reservado normalmente para 'creado'). Es un detalle a revisar.

• ModificarDatos(req, res) — Actualizar (Update)

- Toma el `ID_Planeta` desde los parámetros de la ruta (`req.params`) y los nuevos datos desde `req.body`.
- Ejecuta `UPDATE Planetas SET Nombre=?, Descripcion=?, Dis=? WHERE ID_Planeta=?`.
- Si falla → responde 500.
- Si no hubo cambios (`this.changes === 0`) → responde 401.

Nota: el código 401 significa 'No autorizado'; sería más apropiado usar 404 (no encontrado) o 400 (sin cambios), ya que no es un problema de autenticación.

- Si tiene éxito → responde 201 confirmando la modificación.

• EliminarDatos(req, res) — Eliminar (Delete)

- Referenciada en `Rutas.routes.js` (`Rutas.delete('/Eliminar/:ID_Planeta', ...)`) pero no aparece en las imágenes provistas.
- Debería seguir la misma lógica que las anteriores: tomar `ID_Planeta` de `req.params` y ejecutar `DELETE FROM Planetas WHERE ID_Planeta = ?`.

5.4 Rutas.routes.js — Definición de endpoints

Define el enrutador de Express y conecta cada ruta HTTP con su función del Controller:

Método	Endpoint	Función	Acción
POST	/registro	RegistroDatos	Crear un planeta
GET	/verTabla	VerDatos	Ver todos los planetas
PUT	/Modificar/:ID_Planeta	ModificarDatos	Modificar un planeta por ID
DELETE	/Eliminar/:ID_Planeta	EliminarDatos	Eliminar un planeta por ID

Al final, `module.exports = Rutas` exporta el enrutador para que `index.js` lo pueda montar en la aplicación.

6. Cómo usar la API (ejemplos)

Usando curl, Postman o Thunder Client, con el servidor corriendo en `http://localhost:3001`:

Registrar un planeta

```
curl -X POST http://localhost:3001/registro \
-H "Content-Type: application/json" \
-d '{"Nombre":"Marte","Descripcion":"Planeta rojo","Dis":"228 millones km"}'
```

Ver todos los planetas

```
curl http://localhost:3001/verTabla
```

Modificar un planeta (ID 1)

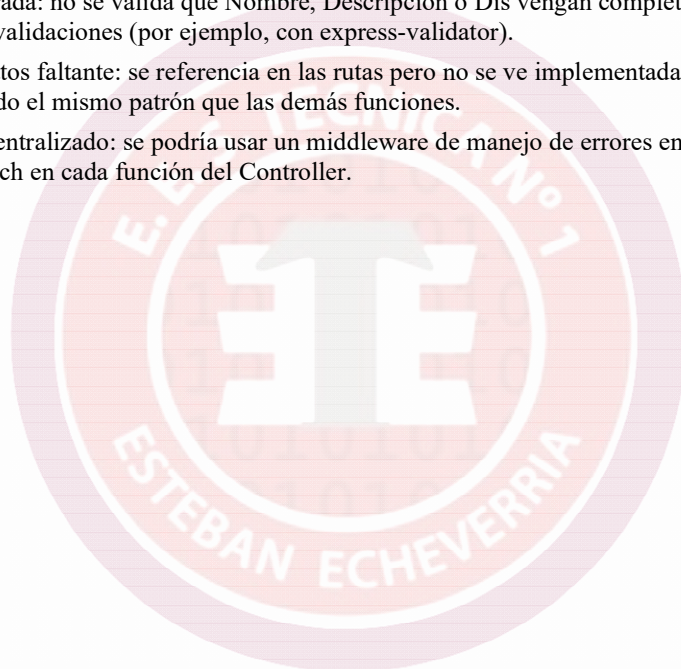
```
curl -X PUT http://localhost:3001/Modificar/1 \  
-H "Content-Type: application/json" \  
-d '{"Nombre":"Marte","Descripcion":"Planeta rojo actualizado","Dis":"227.9 millones km"}'
```

Eliminar un planeta (ID 1)

```
curl -X DELETE http://localhost:3001/Eliminar/1
```

7. Observaciones y posibles mejoras

- Códigos de estado HTTP: revisar el uso de 201 para lecturas (VerDatos); lo correcto sería 200. En ModificarDatos, usar 404 o 400 en vez de 401 cuando no hay cambios.
- Validaciones de entrada: no se valida que Nombre, Descripcion o Dis vengan completos antes de insertarlos; convendría agregar validaciones (por ejemplo, con express-validator).
- Función EliminarDatos faltante: se referencia en las rutas pero no se ve implementada en el material provisto; debe completarse siguiendo el mismo patrón que las demás funciones.
- Manejo de errores centralizado: se podría usar un middleware de manejo de errores en index.js para evitar repetir la misma lógica try/catch en cada función del Controller.



GAREIS PABLO

DOCENTE TÉCNICO